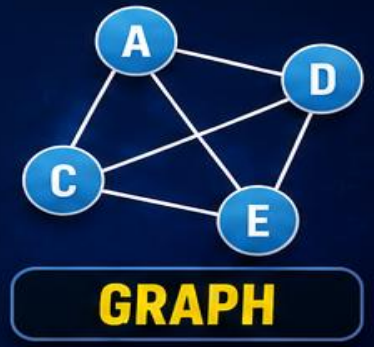
INTRODUCTION TO DATA STRUCTURE

THE FOUNDATION OF EFFICIENT PROGRAMMING!



EASY EXPLANATION | BASIC CONCEPTS | REAL LIFE EXAMPLES

PERFE BEGIN

INFOMAX COMPUTER ACADEMY

What is Data?

- Data is raw facts and figures
- Can be numbers, text, images, etc.
- Example: 10, “infomax", 98.5

What is a Data Structure?

- A way of organizing and storing data
- Enables efficient access and modification
- Helps in better memory utilization

Need for Data Structures

- Efficient data management
- Faster access and processing
- Reduces time and space complexity
- Essential for large applications

Types of Data Structures

- **Primitive Data Structures**
  - int, float, char, pointer
- **Non-Primitive Data Structures**
  - Linear & Non-linear

Linear Data Structures

- Elements arranged sequentially Easy to traverse Examples:
  - Array
  - Linked List
  - Stack
  - Queue

Non-Linear Data Structures

- Elements arranged hierarchically Complex relationships Examples:
  - Tree
  - Graph

Operations on Data Structures

- Insertion
- Deletion
- Traversal
- Searching
- Sorting

Characteristics of Data Structures

- Efficiency
- Reusability
- Abstraction
- Scalability

Efficiency of an Algorithm

- Algorithm efficiency measures performance
- Helps compare different algorithms
- Important for large-scale problems

What is an Algorithm?

- Step-by-step procedure to solve a problem
- Takes input and produces output

Why Efficiency Matters

- Saves time and resources
- Improves performance
- Essential for real-time systems

Types of Efficiency

- Time Efficiency
- Space Efficiency

Time Complexity

- Measures execution time of an algorithm
- Depends on input size (n)
- Expressed using Big-O notation

Common Time Complexities

- O(1) – Constant time
- O(log n) – Logarithmic
- O(n) – Linear
- O(n log n) – Linearithmic
- $O(n^2)$ – Quadratic

Space Complexity

- Measures memory usage
- Includes:
  - Fixed part
  - Variable part

Asymptotic Notations

- Big-O (O) → Worst case
- Big-Omega (Ω) → Best case
- Big-Theta (Θ) → Average case

Factors Affecting Efficiency

- Input size
- Algorithm design
- Hardware & system environment

WHAT IS ARRAY EXPLAINED

EASY EXPLANATION!

- CONCEPT
- EXAMPLES
- PROGRAMS
- EASY TO UNDERSTAND

FOR BEGINNERS

INFOMAX COMPUTER ACADEMY

Introduction

- Collection of elements of same data type
- Stored in contiguous memory locations
- Accessed using index
- Index starts from 0

Characteristics of Arrays

- Fixed size
- Homogeneous elements
- Fast access (O(1))
- Easy traversal

Single Dimensional Array

- One row of elements
- Example in Python:

```
arr = [10, 20, 30, 40]
print(arr[2])  # Output: 30
```

Multidimensional Array

- Array of arrays
- Example: 2D array (Matrix)

```
matrix = [
    [1, 2, 3],
    [4, 5, 6]
]
print(matrix[1][2])  # Output: 6
```

Row Major Order

- Row-wise storage
- Formula:
- $Address = Base + [(i \times n) + j] \times size$
- Used in languages like C

Column Major Order

- Column-wise storage
- Formula:
- $Address = Base + [(j \times m) + i] \times size$
- Used in languages like Fortran

Applications of Arrays

- Storing lists of data
- Matrix operations
- Searching & sorting
- Implementing other data structures

Sparse Matrix

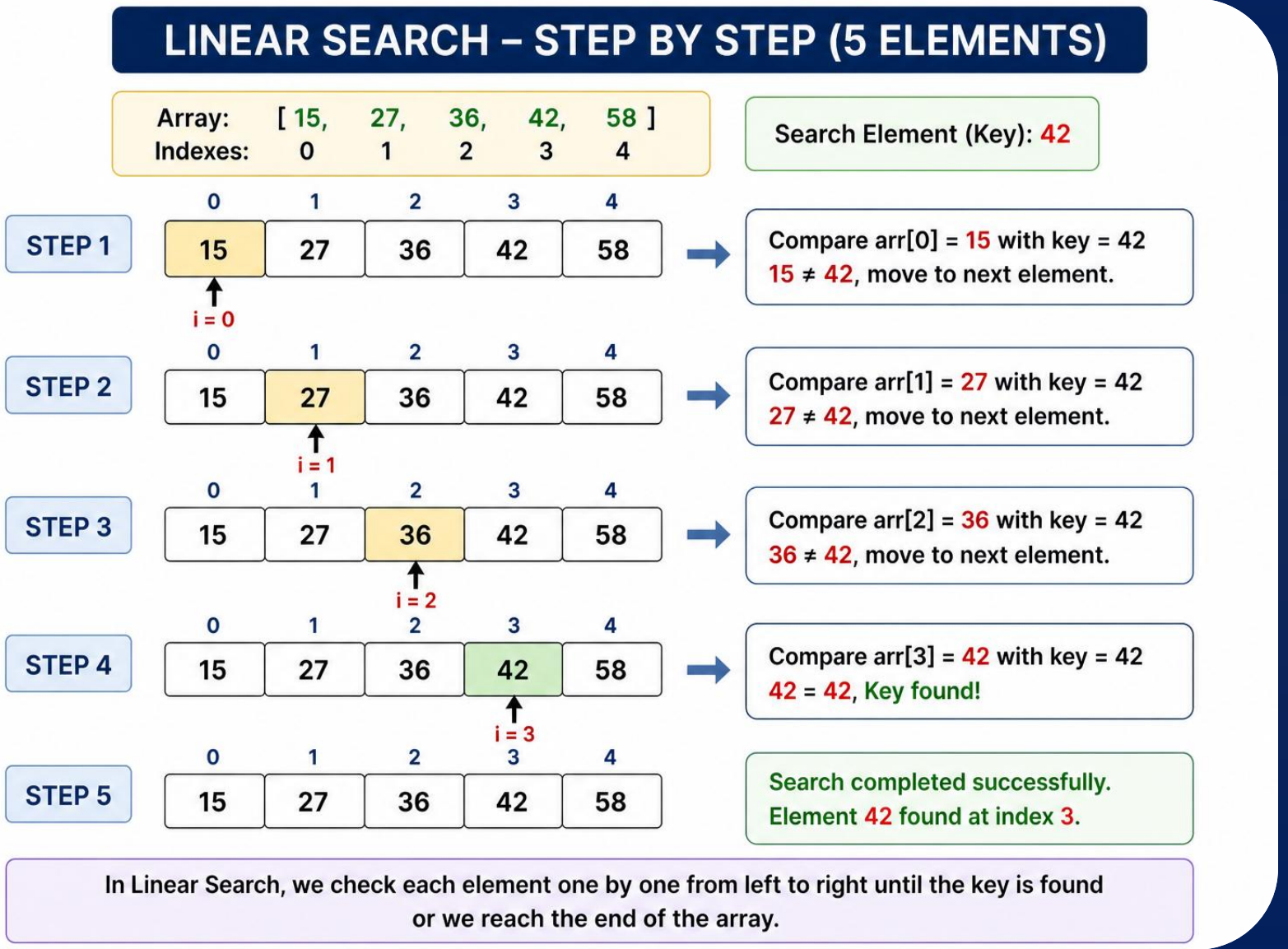
- Matrix with mostly zero values
- Saves memory by storing only non-zero elements
- Example:

0 0 5
0 0 0
3 0 0

Searching in Array

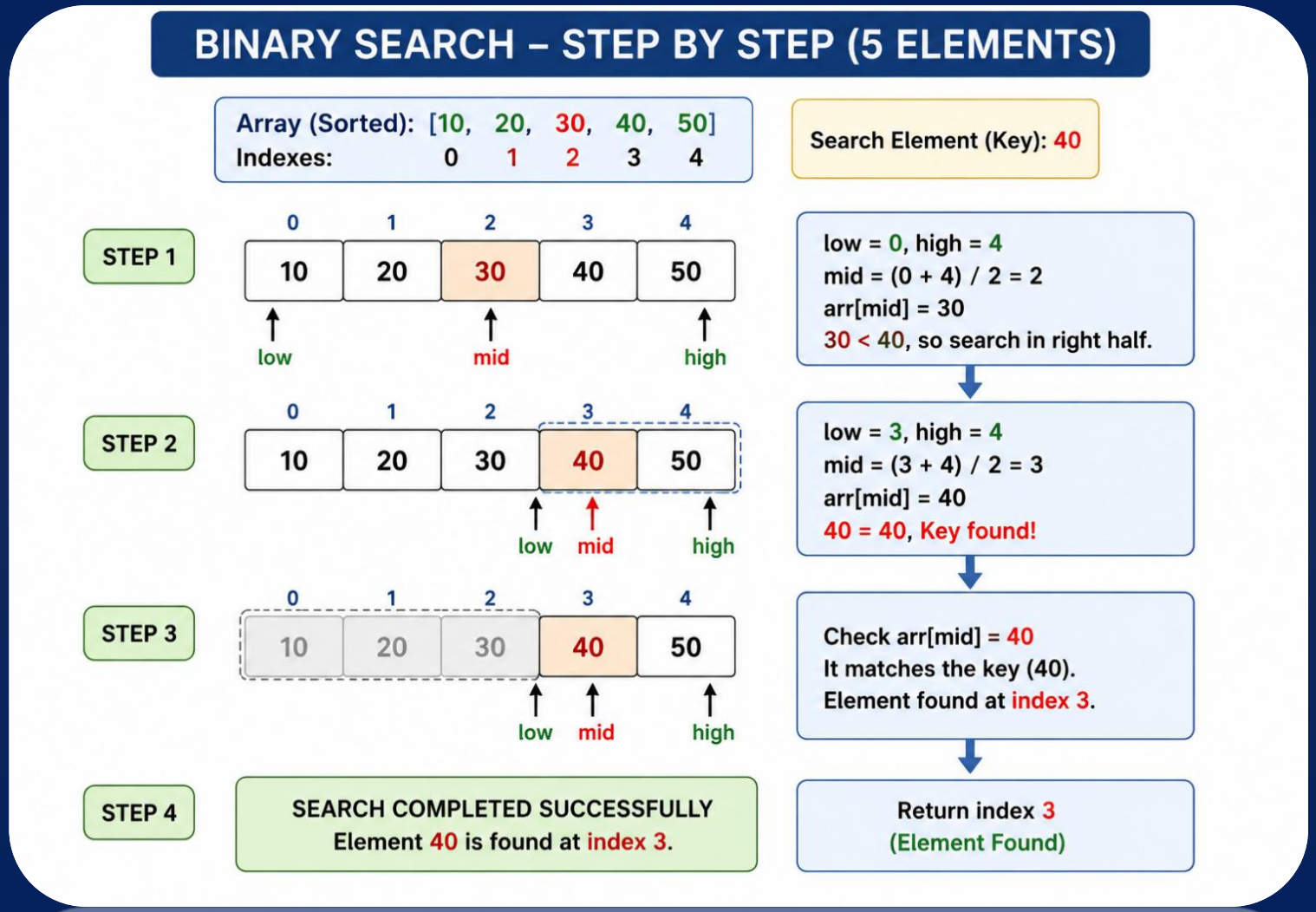
- Linear Search
- Binary Search (Sorted Array)

Linear Search




```
def linear_search(arr, key):
    for i in range(len(arr)):
        if arr[i] == key:
            return i
    return -1
```

Binary Search (Sorted Array)




```
def binary_search(arr, key):
    low, high = 0, len(arr)-1
    while low <= high:
        mid = (low + high)//2
        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1
    return -1
```

Sorting in Array

- Bubble Sort
- Selection Sort
- Insertion Sort

Bubble Sort

BUBBLE SORT – STEP BY STEP (5 ELEMENTS)

Array to be Sorted:

[64, 34, 25, 12, 22]

Indexes: 0 1 2 3 4

Bubble Sort Concept:
Compare adjacent elements.
If left element is greater, swap them.
After each pass, the largest unsorted element comes at the end.

PASS 1 (n-1 = 4 comparisons)	Compare and Swap	After each comparison
Step 1 Compare index 0 & 1	Compare 64 and 34 → 64 > 34, swap 34 64 25 12 22	Largest element (64) moved to the right.
Step 2 Compare index 1 & 2	Compare 64 and 25 → 64 > 25, swap 34 25 64 12 22	Next largest (64) moves further right.
Step 3 Compare index 2 & 3	Compare 64 and 12 → 64 > 12, swap 34 25 12 64 22	64 moves to 4th position.
Step 4 Compare index 3 & 4	Compare 64 and 22 → 64 > 22, swap 34 25 12 22 64	64 moves to last position. End of Pass 1 → 64 is in its correct position.

Array after Pass 1: [34, 25, 12, 22, 64]

PASS 2 (n-2 = 3 comparisons)	Compare and Swap	After each comparison
Step 1 Compare index 0 & 1	Compare 34 and 25 → 34 > 25, swap 25 34 12 22 64	Largest among first 4 elements (34) moves to right.
Step 2 Compare index 1 & 2	Compare 34 and 12 → 34 > 12, swap 25 12 34 22 64	34 moves to 3rd position.
Step 3 Compare index 2 & 3	Compare 34 and 22 → 34 > 22, swap 25 12 22 34 64	34 moves to 4th position.

Array after Pass 2: [25, 12, 22, 34, 64]

PASS 3 (n-3 = 2 comparisons)	Compare and Swap	After each comparison
Step 1 Compare index 0 & 1	Compare 25 and 12 → 25 > 12, swap 12 25 22 34 64	Largest among first 3 elements (25) moves to right.
Step 2 Compare index 1 & 2	Compare 25 and 22 → 25 > 22, swap 12 22 25 34 64	25 moves to 3rd position.

Array after Pass 3: [12, 22, 25, 34, 64]

PASS 4 (n-4 = 1 comparison)	Compare and Swap	After comparison
Step 1 Compare index 0 & 1	Compare 12 and 22 → 12 < 22, no swap 12 22 25 34 64	No swap needed. Array is sorted.

Array after Pass 4: [12, 22, 25, 34, 64]

Sorting Completed!
Final Sorted Array: [12, 22, 25, 34, 64]

Total Comparisons:
4 + 3 + 2 + 1 = 10

Bubble Sort : Code

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n-i-1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
```

Selection Sort

SELECTION SORT – STEP BY STEP (5 ELEMENTS)

Array to be Sorted: [64, 25, 12, 34]

Indexes: 0 1 2 3 4

Selection Sort Concept:
Find the smallest element in the unsorted part and swap it with the first element of that part. Then, move the boundary one position to the right.

Pass	Step	Array (with Index)	Explanation	Result
PASS 1 (i = 0) Place the smallest at index 0	Step 1	0: 64, 1: 25, 2: 12, 3: 22, 4: 34	Find smallest in [64, 25, 12, 22, 34] Smallest = 12 (index 2)	Min = 12
	Step 2	0: 12, 1: 25, 2: 64, 3: 22, 4: 34	Swap smallest (12) with first element (64)	12 placed at index 0
		Array after Pass 1: [12, 25, 64, 22, 34]		
PASS 2 (i = 1) Place the smallest at index 1	Step 1	0: 12, 1: 25, 2: 64, 3: 22, 4: 34	Find smallest in [25, 64, 22, 34] Smallest = 22 (index 3)	Min = 22
	Step 2	0: 12, 1: 22, 2: 64, 3: 25, 4: 34	Swap smallest (22) with element at index 1 (25)	22 placed at index 1
		Array after Pass 2: [12, 22, 64, 25, 34]		
PASS 3 (i = 2) Place the smallest at index 2	Step 1	0: 12, 1: 22, 2: 64, 3: 25, 4: 34	Find smallest in [64, 25, 34] Smallest = 25 (index 3)	Min = 25
	Step 2	0: 12, 1: 22, 2: 25, 3: 64, 4: 34	Swap smallest (25) with element at index 2 (64)	25 placed at index 2
		Array after Pass 3: [12, 22, 25, 64, 34]		
PASS 4 (i = 3) Place the smallest at index 3	Step 1	0: 12, 1: 22, 2: 25, 3: 64, 4: 34	Find smallest in [64, 34] Smallest = 34 (index 4)	Min = 34
	Step 2	0: 12, 1: 22, 2: 25, 3: 34, 4: 64	Swap smallest (34) with element at index 3 (64)	34 placed at index 3
		Array after Pass 4: [12, 22, 25, 34, 64]		

SORTING COMPLETED!
Final Sorted Array: [12, 22, 25, 34, 64]

Total Comparisons:
(n-1) + (n-2) + (n-3) + (n-4)
= 4 + 3 + 2 + 1 = 10

In Selection Sort, we repeatedly select the minimum element from the unsorted part and put it at the beginning of that part.

Selection Sort

```
def selection_sort(arr):
    for i in range(len(arr)):
        min_idx = i
        for j in range(i+1, len(arr)):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
```

Insertion Sort

INSERTION SORT – STEP BY STEP (5 ELEMENTS)

Array to be Sorted: [64, 25, 12, 34]

Indexes: 0 1 2 3 4

Insertion Sort Concept:
Start from index 1 (second element).
Compare the key with elements on its left.
Shift greater elements one position to the right.
Insert the key in its correct position.

Pass	Step	Array (with Index)	Explanation	Key & Position
PASS 1 (i = 1) Insert element at index 1 in sorted part [0...i-1]	Initial	0:64 1:25 2:12 3:34 4:22	Sorted part: [64] Unsorted part: [25, 12, 34, 22]	Key = 25
	Compare	0:64 1:25 2:12 3:34 4:22	Compare key (25) with 64 25 < 64, so shift 64 right	25 < 64
	Shift	0: 1:64 2:12 3:34 4:22	Shift 64 to index 1	64 shifted right
	Insert	0:25 1:64 2:12 3:34 4:22	Insert key (25) at index 0 Sorted part: [25, 64]	25 inserted at index 0
Array after Pass 1: [25, 64, 12, 34, 22]				
PASS 2 (i = 2) Insert element at index 2 in sorted part [0...i-1]	Initial	0:25 1:64 2:12 3:34 4:22	Sorted part: [25, 64] Unsorted part: [12, 34, 22]	Key = 12
	Compare 1	0:25 1:64 2:12 3:34 4:22	Compare key (12) with 64 12 < 64, so shift 64 right	12 < 64
	Shift 1	0: 1:25 2:64 3:34 4:22	Shift 64 to index 2	64 shifted right
	Compare 2	0:25 1:64 2:12 3:34 4:22	Compare key (12) with 25 12 < 25, so shift 25 right	12 < 25
	Shift 2	0: 1:25 2:64 3:34 4:22	Shift 25 to index 1	25 shifted right
	Insert	0:12 1:25 2:64 3:34 4:22	Insert key (12) at index 0 Sorted part: [12, 25, 64]	12 inserted at index 0
Array after Pass 2: [12, 25, 64, 34, 22]				

PASS 3 (i = 3) Insert element at index 3 in sorted part [0...i-1]	Initial	0:12 1:25 2:64 3:34 4:22	Sorted part: [12, 25, 64] Unsorted part: [34, 22]	Key = 34
	Compare 1	0:12 1:25 2:64 3:34 4:22	Compare key (34) with 64 34 < 64, so shift 64 right	34 < 64
	Shift	0:12 1:25 2:64 3:64 4:22	Shift 64 to index 3	64 shifted right
	Compare 2	0:12 1:25 2:34 3:64 4:22	Compare key (34) with 25 34 > 25, stop shifting	34 > 25
	Insert	0:12 1:25 2:34 3:64 4:22	Insert key (34) at index 2 Sorted part: [12, 25, 34, 64]	34 inserted at index 2
Array after Pass 3: [12, 25, 34, 64, 22]				
PASS 4 (i = 4) Insert element at index 4 in sorted part [0...i-1]	Initial	0:12 1:25 2:34 3:64 4:22	Sorted part: [12, 25, 34, 64] Unsorted part: [22]	Key = 22
	Compare 1	0:12 1:25 2:34 3:64 4:22	Compare key (22) with 64 22 < 64, so shift 64 right	22 < 64
	Shift 1	0:12 1:25 2:34 3:64 4:64	Shift 64 to index 4	64 shifted right
	Compare 2	0:12 1:25 2:34 3:22 4:64	Compare key (22) with 34 22 < 34, so shift 34 right	22 < 34
	Shift 2	0:12 1:25 2:22 3:34 4:64	Shift 34 to index 3	34 shifted right
	Compare 3	0:12 1:25 2:34 3:34 4:64	Compare key (22) with 25 22 < 25, so shift 25 right	22 < 25
	Shift 3	0:12 1:22 2:25 3:34 4:64	Shift 25 to index 2	25 shifted right
	Compare 4	0:12 1:22 2:25 3:34 4:64	Compare key (22) with 12 22 > 12, stop shifting	22 > 12
	Insert	0:12 1:22 2:25 3:34 4:64	Insert key (22) at index 1 Sorted part: [12, 22, 25, 34, 64]	22 inserted at index 1
Array after Pass 4: [12, 22, 25, 34, 64]				

SORTING COMPLETED!
Final Sorted Array: [12, 22, 25, 34, 64]

Total Comparisons: 10
Total Shifts: 13

In Insertion Sort, we build the sorted array one element at a time by inserting each element into its correct position in the sorted part.

Insertion Sort

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i-1
        while j >= 0 and key < arr[j]:
            arr[j+1] = arr[j]
            j -= 1
        arr[j+1] = key
```

Time Complexity

- Linear Search → O(n)
- Binary Search → O(log n)
- Bubble Sort → $O(n^2)$
- Selection Sort → $O(n^2)$
- Insertion Sort → $O(n^2)$

Thank You

Address: G. R. Complex Preetam Nagar, Dhoomanganj
Prayagraj Uttar Pradesh 211011

Mobile No.: 9598948810,8874588766,9532878816
Website : www.infomax.org.in